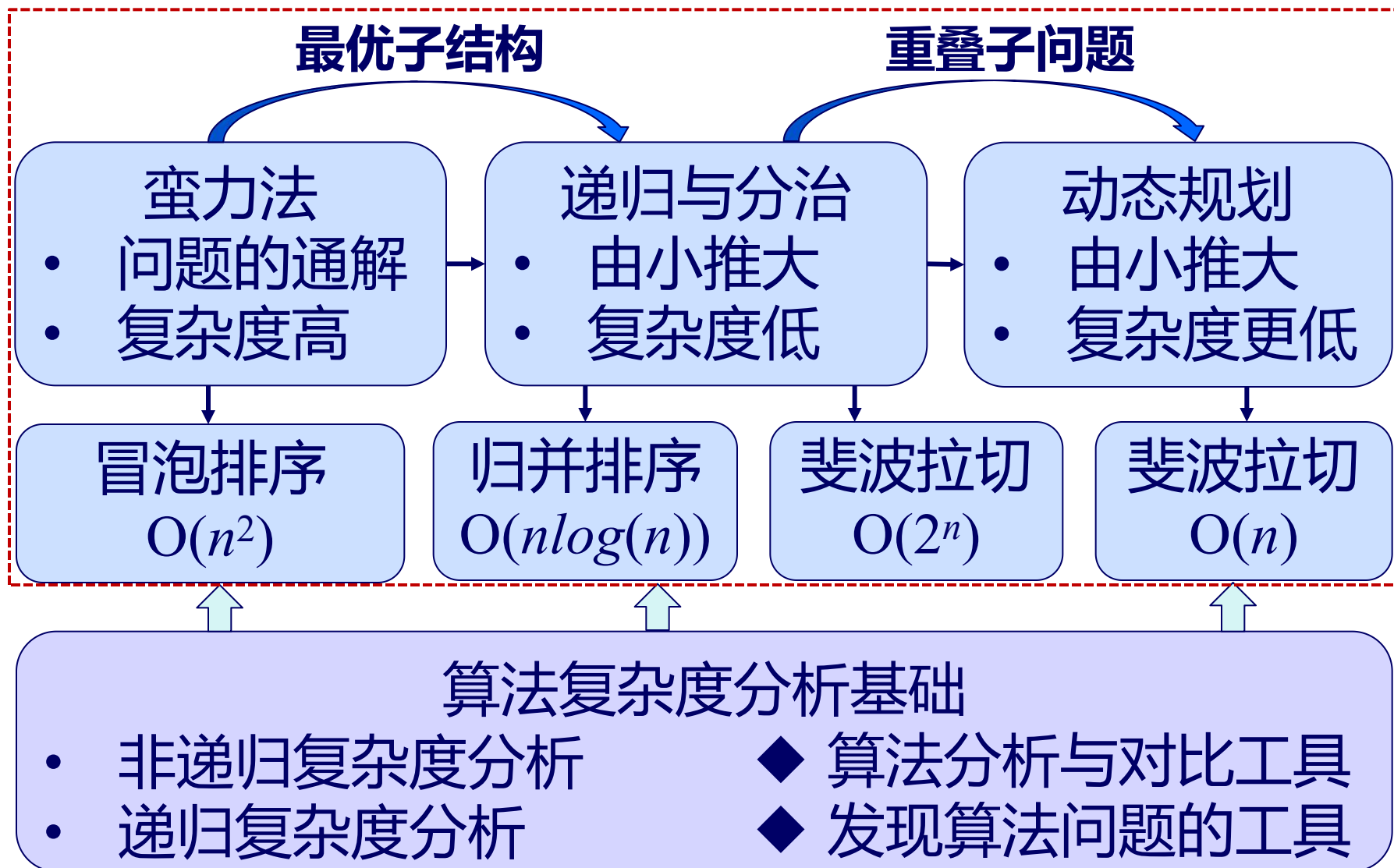
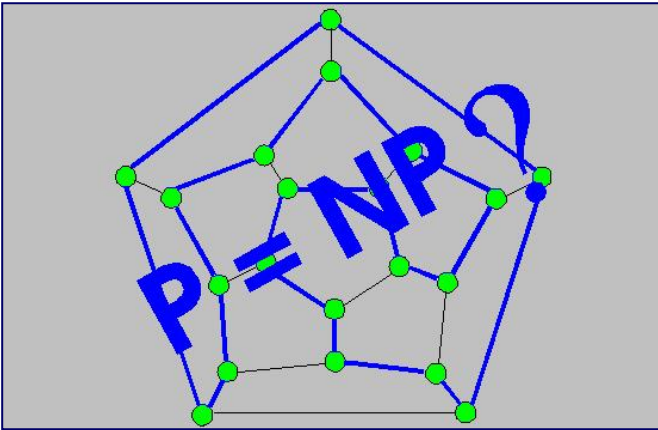


知识回顾与梳理



6-贪心算法



罗建超

长聘副教授、博士研究生导师

算法分析与设计

Introduction to the Analysis and Design of Algorithms

November 14, 2025

目录

1. 贪心算法的基本思想
2. 贪心算法最优解条件
3. 贪心算法的最优性证明
4. 贪心算法求解问题案例
5. 贪心算法得不到最优情况的处理
6. 总结与开放性讨论

贪心算法的基本思想

- 当一个问题具有最优子结构和重复子问题性质时，可用动态规划方法求解。
- 例如

假设有5种货币，它们的面值分别为二元、一元、五角，二角和一角，现要找给顾客四元八角钱，问如何选取个数最少的货币？

贪心算法的基本思想

$d(i)$ 来表示需要凑出 i 的总价值需要的最少货币数量;

1、 $i = 0, d(0) = 0$;

2、 $i = 1, d(1) = d(0) + 1 = 1$ (≤ 1 角, 只有1角) ;

3、 $i = 2, d(2) = \min\{d(1) + 1, d(0) + 1\} = 1$; (≤ 2 角, 有2角和1角) ;

4、 $i = 3, d(3) = \min\{d(2) + 1, d(1) + 1\} = 2$; (≤ 3 角, 有2角和1角) ;

5、 $i = 4, d(4) = \min\{d(3) + 1, d(2) + 1\} = 2$; (≤ 4 角, 有2角和1角) ;

6、 $i = 5, d(5) = \min\{d(4) + 1, d(3) + 1, d(0) + 1\} = 1$; (≤ 5 角, 有5角、2角和1角)

....

因此 $d(i) = \min\{d(i - j) + 1 \mid j \in \{\text{货币面值集合}\}\}$;

具体来说 $d(i) = \min\{d(i - j) + 1 \mid j \in \{1, 2, 5, 10, 20\}\}$;

贪心算法的基本思想

■ 经常性做法

假设有5种货币，它们的面值分别为二元、一元、五角、二角和一角，现要找给顾客四元八角钱，问如何选取个数最少的货币？

2个二元，1个五角，1个两角，1个1角。

过程：从大到小，是否为最优解？

如果3种货币，面值分别为八角，五角和一角，现要找1元钱，问上述从大到小过程是否为最优？

贪心算法的基本思想

- 顾名思义，贪心算法总是作出在**当前看来最好的选择**。贪心算法并不从整体最优考虑，它所作出的选择只是在某种意义上的**局部最优选择**。
- 贪心算法**不能对所有问题都得到整体最优解**，但对一些问题它能产生整体最优解。
- 在一些情况下，即使贪心算法不能得到整体最优解，其最终结果却是**最优解的很好近似**。

贪心算法的基本思想

- 贪心算法中，较大子问题的解恰好包含了较小子问题的解作为子集，这与动态规划算法设计中的优化原则本质上是一致的。
- 动态规划算法在每步决策时，考虑了它的所有子问题的函数值，然后从中选出最优的结果。
- 贪心算法在每步决策时，不考虑子问题的计算结果，而是根据当时情况采取“只顾眼前”的贪心策略决定取舍。

贪心算法最优解条件

- 可以用贪心算法求解**得到最优解**的问题，一般具有**2个**重要的性质：
 - **最优子结构性质**
 - **贪心选择性质**

贪心算法最优解条件

- **最优子结构性质**

- 当一个问题的最优解可以由其子问题的最优解推出时，称此问题具有最优子结构性质。问题的最优子结构性质是该问题可用动态规划算法或贪心算法求解的关键特征。

贪心算法最优解条件

- **贪心选择性质**

- 贪心选择性质是指所求问题的整体最优解可以通过一系列**局部最优的选择**，即贪心选择来达到。
- 贪心算法以**自顶向下**的方式进行，每作一次贪心选择就将所求问题简化为规模更小的子问题。

贪心算法最优性证明

- 针对具体问题不同，贪心策略的选择可能有多种，如何**选择合适的贪心策略**并证明该策略的正确性是贪心算法设计中的一个关键问题。
- 一般可以通过对**算法步数的归纳**或通过对**问题规模的归纳**来证明贪心法的最优性。

贪心算法求解问题案例

■ 活动安排问题

有 n 个活动申请使用同一个礼堂，每项活动有一个开始时间和一个截止时间，如果任何两个活动不能同时举行，问如何选择这些活动，从而使得被安排的活动数量达到最多？

设 $S = \{a_1, a_2, \dots, a_n\}$ 为活动的集合， s_i 和 f_i 分别为活动 a_i 的开始和截止时间， $i = 1, 2, \dots, n$ 。定义

活动 i 与 j 相容（都能安排）： $s_i \geq f_j$ 或 $s_j \geq f_i, i \neq j$

求 S 最大的两两相容的活动子集。

贪心算法求解问题案例

■ 活动安排问题—直觉贪心策略

策略1：尽早占用

把活动按照开始时间从小到大排序，使得 $s_1 \leq s_2 \leq \dots \leq s_n$ ，然后从前向后挑选，只要与前面选择的活動相容，便将这项活动选入最大相容集合 A 。

策略2：时间占用少的活动先安排

计算每个活动占用时间 $f_i - s_i$ ，按照占用时间从小到大对活动排序，使得 $f_1 - s_1 \leq f_2 - s_2 \leq \dots \leq f_n - s_n$ ，然后从前向后挑选，只要与前面活动相容，便将这项活动选入最大相容集合 A 。

策略3：早完成的活動先安排

把活动按照截止时间从小到大排序，使得 $f_1 \leq f_2 \leq \dots \leq f_n$ ，然后从前向后挑选，只要与前面选择的活動相容，便将这项活动选入最大相容集合 A 。

贪心算法求解问题案例

■ 活动安排问题—直觉贪心策略

策略1 (**早到早开始**) 不正确。

反例: $S = \{1, 2, 3\}$, $a_1 = \langle 0, 20 \rangle$, $a_2 = \langle 2, 5 \rangle$, $a_3 = \langle 8, 15 \rangle$ 。

活动 1 开始后, 与 2, 3 都冲突, 只能有 1 个活动。显然最大为 2 个活动。

策略2 (**占时间少先开始**) 不正确。

反例: $S = \{1, 2, 3\}$, $a_1 = \langle 0, 8 \rangle$, $a_2 = \langle 7, 9 \rangle$, $a_3 = \langle 8, 15 \rangle$ 。

活动 2 开始后, 与 1, 3 都冲突, 只能有 1 个活动。显然最大为 2 个活动。

策略 3 最优?

贪心算法求解问题案例

■ 活动安排问题—贪心策略正确性证明

假设 S 是活动安排的集合，且按照结束时间由小到大排好

假设 A 为某个最优活动安排，且 A 中的活动也按照结束时间由小到大排好

首先证明：第1个活动一定在包含在某个最优安排活动集合中；

证明：假设 k 是 A 中的第一个活动，如果 $k = 1$ ，得证。如果 $k > 1$ 。由于 $f_1 \leq f_k$ ，且活动 k 与 A 中的其它活动相互兼容，因此活动 1 也与 A 中除去 k 外的其它活动兼容。由于 A 是最优安排，因此 $B = \{1\} \cup A \setminus \{k\}$ 也是最优安排，得证。

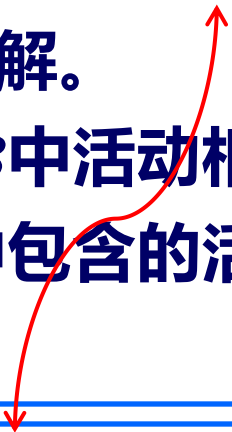
贪心算法求解问题案例

■ 活动安排问题—贪心策略正确性证明

证明了第一步后，证明就简化为 S 中所有与活动 1 相容的活动进行活动安排的问题。

命题：如果 A^* 是 S 中所有与活动 1 相容的活动集合 $\{i \in S: s_i \geq f_1\}$ 的最优解， $A = A^* \cup \{1\}$ 是原问题包含活动 1 的最优解。

证明：如果在集合 $\{i \in S\}$ 中存在子集 $B = B^* \cup \{1\}$ ， B 中活动相容且比 A 包含的活动更多，则 B^* 中的集合一定比 A^* 中包含的活动更多，与 A^* 为最优矛盾，因此不存在这样的集合。



因此，每一次贪心选择后，就转换为了一个与**原问题相同形式**的**更小问题**，由数学归纳可知，贪心选择策略正确。

贪心算法求解问题案例

■ 活动安排问题

```
void GreedySelector(int n, Type s[], Type f[], bool A[]) {  
    A[1] = true;  
    int j = 1;  
    for (int i = 2; i ≤ n; i++) {  
        if (s[i] ≥ f[j]) { //相容条件  
            A[i] = true;  
            j = i;  
        }  
        else A[i] = false;  
    }  
}
```

各活动起始时间与结束时间分别存储于数组 s 和 f 中, 且 f 中活动按结束时间非递减排序, 即 $f_1 \leq f_2 \leq \dots \leq f_n$ 。

时间复杂度 $O(n)$
预排序 $O(n \log n)$

该算法的贪心选择的意义是使剩余的可安排时间段极大化, 以便安排尽可能多的相容活动。

贪心算法求解问题案例

■ 活动安排问题—实例计算过程

设待安排的11个活动的开始时间和结束时间按结束时间的非减序排列如下：

i	1	2	3	4	5	6	7	8	9	10	11
$s[i]$	1	3	0	5	3	5	6	8	8	2	12
$f[i]$	4	5	6	7	8	9	10	11	12	13	14

最终解为： $A = \{1, 4, 8, 11\}$

贪心算法求解问题案例

■ 活动安排问题扩展

飞机停机位安排问题：某航空公司在咸阳机场有10个靠桥停机位和一些远端停机位。为了让乘客少走路，公司希望让尽可能多的飞机安排在靠桥位置停靠。假设所有的飞机都可以安排在任何停机位，问如何安排靠桥位置，使得航班靠桥率最高？

这种安排方式是否是最优的？

反例：2个停机位，5架飞机 $\{(1, 3), (3, 4), (4, 8), (5, 9), (8, 10)\}$ ，前三架安排在1号停机位，则4和5只能安排一架；1,3,5一起，2和4一起，都能安排！

贪心算法求解问题案例

■ 活动安排问题扩展

飞机停机位安排问题：某航空公司在咸阳机场有10个靠桥停机位和一些远端停机位。为了让乘客少走路，公司希望让尽可能多的飞机安排在靠桥位置停靠。已知停机位**有大小之分**，**飞机也有大小之分**，大飞机只能停靠大停机位，小飞机可以停靠大或小停机位。问如何安排靠桥位置，使得航班靠桥率最高？

这种安排方式是否是最优的？

贪心算法求解问题案例

■ 0-1背包问题与背包问题

0-1背包问题：给定 n 个物品和一个背包。物品 i 的重量为 w_i ，其价值为 v_i ，背包的容量为 c 。问应如何选择装入背包中的物品，使得装入背包中的物品总价值最大？

说明：同一物品**不能装入多次**，也**不能装入物品的一部分**。

背包问题：与0-1背包问题类似，所不同的是在选择物品 i 装入背包时，可以选择物品的一部分，而**不一定要全部装入背包**。

贪心算法求解问题案例

■ 0-1背包问题与背包问题

0-1背包的**问题描述**为：给定 $c > 0, w_i > 0, v_i > 0, 1 \leq i \leq n$,

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq c \\ x_i \in \{0,1\}, 1 \leq i \leq n \end{cases} \quad \max \sum_{i=1}^n v_i x_i$$

背包的**问题描述**为：给定 $c > 0, w_i > 0, v_i > 0, 1 \leq i \leq n$,

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq c \\ 0 \leq x_i \leq 1, 1 \leq i \leq n \end{cases} \quad \max \sum_{i=1}^n v_i x_i$$

贪心算法求解问题案例

■ 背包问题的贪心算法

- 首先计算每种物品单位重量的价值 v_i/w_i ;
- 然后，依贪心选择策略，将尽可能多的单位重量价值最高的物品装入背包。若将这种物品全部装入背包后，背包内的物品总重量未超过 c ，则选择单位重量价值次高的物品并尽可能多地装入背包；
- 依此策略一直地进行下去，直到背包装满为止。

贪心算法求解问题案例

■ 背包问题的贪心算法

```
void Knapsack(int n, float M, float v[], float w[], float x[]) {  
    Sort(n, v, w); //按单位重量价值排序  
    int i;  
    for (i = 1; i <= n; i++) x[i]=0;  
    float c = M; //剩余容量  
    for (i = 1; i <= n; i++) {  
        if (w[i] > c) break; //装不下  
        x[i]=1;  
        c -= w[i];  
    }  
    if (i <= n) x[i] = c/w[i]; //把余量装满  
}
```

算法Knapsack的主要计算时间在于将各种物品依其单位重量的价值从大到小排序。因此，算法的计算时间上界为 $O(n\log n)$ 。为了证明算法的正确性，还必须证明背包问题具有贪心选择性质。

贪心算法求解问题案例

■ 背包问题的贪心算法

对于0-1背包问题，贪心选择之所以不能得到最优解是因为在这种情况下，**它无法保证最终能将背包装满**，部分闲置的背包空间使每公斤背包空间的价值降低了。

0,1背包问题反例： $c = 50$ ，物品1-3，（重量，价值）分别为 $(30, 60)$ ， $(25, 40)$ ， $(25, 40)$

贪心：先选物品 $(30, 60)$ ，其它物品就不能选了，价值60，很显然选后两个物品，可以达到80价值。

贪心算法求解问题案例

■ 最优装载问题—问题描述

有一批集装箱要装上一艘载重量为 c 的轮船。其中集装箱 i 的重量为 w_i 。最优装载问题要求确定在装载体积不受限制的情况下，将尽可能多的集装箱装上轮船。

0-1背包问题的特例（约束相同，目标不同，如果物品价值相同则两个问题相同）

$$\begin{cases} \sum_{i=1}^n w_i x_i \leq c \\ x_i \in \{0,1\}, 1 \leq i \leq n \end{cases} \quad \max \sum_{i=1}^n x_i$$

贪心算法求解问题案例

■ 最优装载问题—贪心策略

将集装箱按从轻到重排序，轻者先装。

```
template<class Type>
void Loading(int x[], Type w[], Type c, int n) {
    int* t = new int[n+1]; //按重量记录物品编号
    Sort(w, t, n); //按重量升排序
    for (int i = 1; i <= n; i++) x[i] = 0; //初始化
    for (int i = 1; i <= n && w[t[i]] <= c; i++) {
        x[t[i]] = 1; //能装下就装
        c -= w[t[i]]; //剩余容量减少
    }
}
```

算法Loading的主要计算量在于将集装箱依其重量从小到大排序，故算法所需的计算时间为 $O(n\log n)$ 。

贪心算法求解问题案例

■ 最优装载问题—贪心策略正确性证明

命题：对任何规模为 n (n 是正整数) 的输入，上述贪心法都得到最优解。

思路：对**问题规模**进行归纳。

- (1) 设集装箱已按**重量从小到大排序**，标号为 $1, 2, \dots, n$;
- (2) $k = 1$ ，只有1个集装箱，装($w_1 \leq c$) 或不装($w_1 > c$)，算法得到最优解 I_1 ;
- (3) 假设算法对于规模为 $n-1$ 的输入算法得到最优解，则对于 $N' = \{2, 3, \dots, n\}$ ， $c' = c - w_1$ ，算法得到最优解 I' ;
- (4) 对于规模 n 的输入 $N = \{1, 2, \dots, n\}$ ，其中 $w_1 \leq w_2 \leq \dots \leq w_n$ 。则 **$I = I' \cup I_1$ 为 N 的最优解**。若最优解为 I^* ，且 I^* 中不包含 I_1 ，则用 I_1 替换 I^* 中标号最小的集装箱得到的解 I_1^* 也为最优解。结合(3)可得， $|I \setminus \{I_1\}| \geq |I_1^* \setminus \{I_1\}|$ 。因为 I_1^* 为 N 最优解，所以 $I = I' \cup I_1$ 为最优解。

贪心算法求解问题案例

■ 最优前缀码问题

起源：给出现**频率高**的字符**较短**的编码，出现**频率较低**的字符以**较长**的编码，可以大大**缩短总码长**。

对每一个字符规定一个0,1串作为其代码，并要求**任一字符的代码都不是其它字符代码的前缀**，这种编码称为**前缀码**。编码的**前缀性质能够保证译码方法非常简单**。

定长编码是否有前缀的约束？

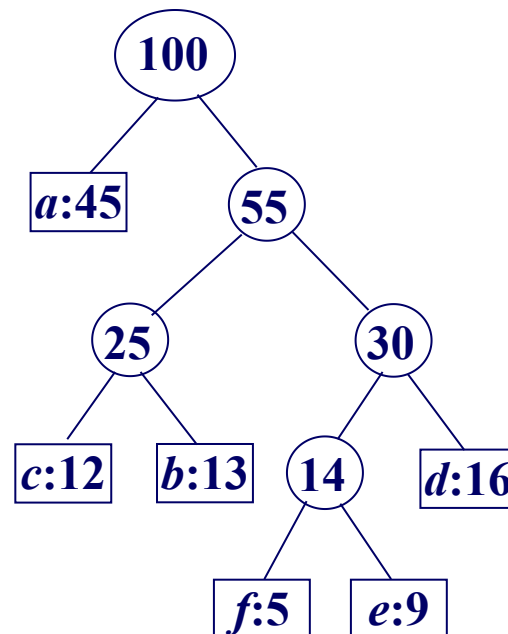
贪心算法求解问题案例

■ 最优前缀码问题—实例分析

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
频次	45	13	12	16	9	5
定长码	000	001	010	011	100	101
变长码	0	101	100	111	1101	1100

定长:
300

变长:
224



贪心算法求解问题案例

■ 最优前缀码问题

前缀编码可以用一颗二叉树表示，树中**叶节点**代表给定字符，字符前缀码可以看作是从**根节点到该字符叶节点的树的路径**。

给定编码字符集 C 及其频率分布 f ，即 C 中任一字符 c 以频率 $f(c)$ 在文件中出现。 C 的一个前缀编码方案对应一个二叉树 T ，字符 c 在 T 中的深度记为 $d_T(c)$ ， $d_T(c)$ 也是字符 c 的前缀码长。编码方案的**总码长**：

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

问题：给定字符集 $C = \{c_1, c_2, \dots, c_n\}$ 以及每个字符频率 $f(c_i)$ ，求关于 C 的一个最优前缀编码（总码长最小）。

贪心算法求解问题案例

■ 最优前缀码问题—Huffman编码

哈夫曼提出了构造最优前缀码的贪心算法，由此产生的编码方案称为**哈夫曼编码**。流程如下：

- a) 哈夫曼算法以字符出现的**频率 f** 为键值构造优先队列 Q ;
- b) 每一次从 Q 中取出 f 值**最小的两棵树**; **//贪心**
- c) 合并取出来的树，将最小的两棵树的频率之和作为新树的频率，将合并后的新树插入 Q ; **//避免前缀**
- d) 重复b)和c) $n-1$ 次，即可。

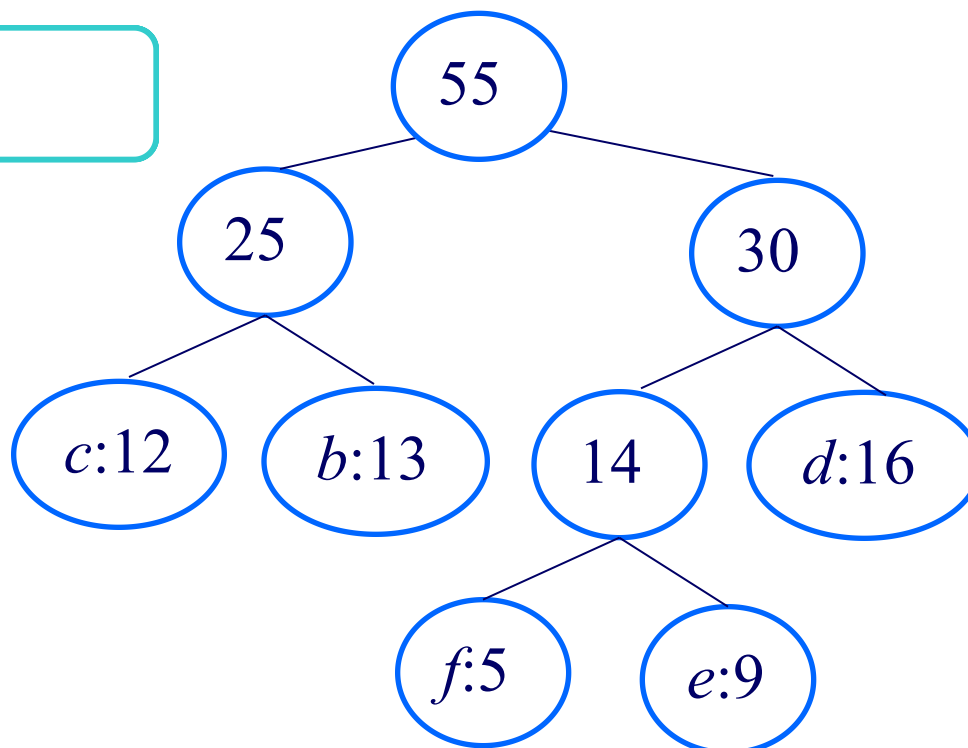
每一次取出2个，插入1个， $n-1$ 次后就可以构造完！

贪心算法求解问题案例

■ 最优前缀码问题—Huffman编码

$Q: \{55\}$

$b: 01;$
 $c: 00;$
 $d: 11;$
 $e: 101;$
 $f: 100;$

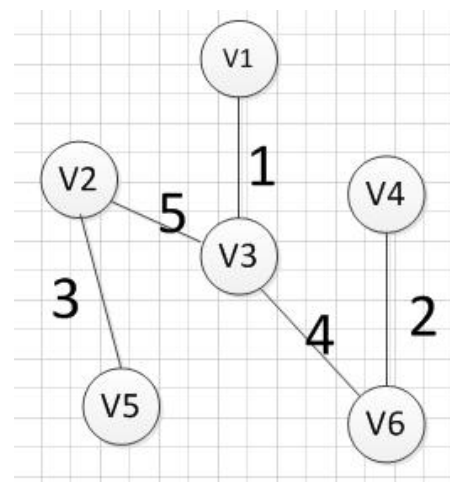
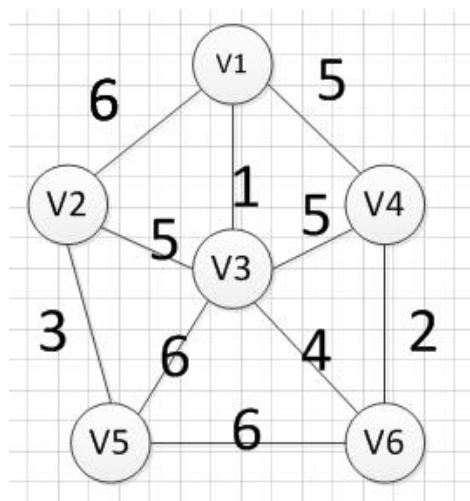


哈夫曼编码是广泛地用于数据文件压缩的十分有效的编码方法。其压缩率通常在20%~90%之间。

贪心算法求解问题案例

■ 最小生成树问题

设 $G = (V, E, W)$ 是无向连通带权图，即一个网络。 E 中每条边 (u, v) 的权为 $w[u][v]$ 。如果 G 的子图 G' 是一棵包含 G 的所有顶点的树，则称 G' 为 G 的生成树。生成树上各边权的总和称为该生成树的耗费。在 G 的所有生成树中，耗费最小的生成树称为 G 的**最小生成树**。



贪心算法求解问题案例

■ 最小生成树问题

网络的最小生成树在实际中有广泛应用。

例如，在设计**通信网络**时，用图的顶点表示城市，用边 (u, v) 的权 $w[u][v]$ 表示建立城市 u 和城市 v 之间的通信线路所需的费用，则最小生成树就给出了建立通信网络的最经济的方案。

问题：给定无向联通带权图 $G(V, E, W)$ ，求 $W(T)$ 最小的生成树

贪心算法求解问题案例

■ 最小生成树问题

最小生成树性质：

设 $G = (V, E, W)$ 是连通带权图， U 是 V 的真子集。如果 $(u, v) \in E$ ，且 $u \in U$ ， $v \in \{V \setminus U\}$ ，且在所有这样的边中， (u, v) 的权 $w[u][v]$ 最小，那么一定存在 G 的一棵最小生成树，它以 (u, v) 为其中一条边。这个性质有时也称为 MST (Most Small Tree) 性质。

贪心算法求解问题案例

■ 最小生成树问题

最小生成树性质证明：

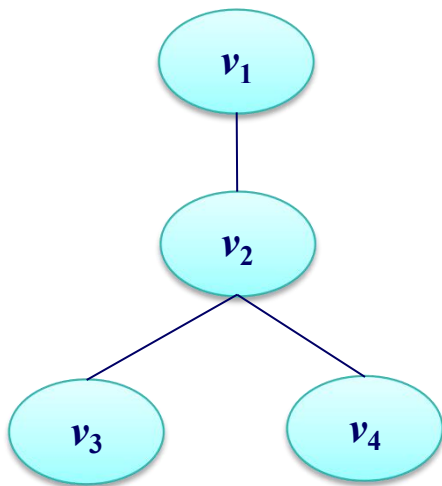


图1 最小生成树1

如果 (v_1, v_4) 是原图中最短的边，且不在最小生成树中，那么删除到 v_4 的边，添加 (v_1, v_4) ，得到的生成树，一定比原来更小。

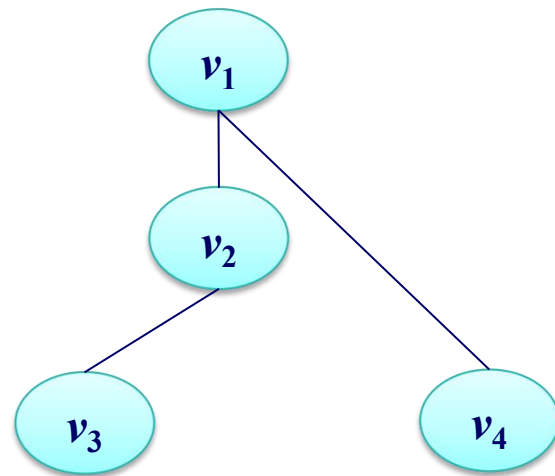


图2 最小生成树2

贪心算法求解问题案例

■ 最小生成树问题

Prim
思想

最小生成树性质理解：

理解1：与**某个节点**相关的权值最小的边，一定在某个最小生成树中，证明同MST证明过程。

理解2：权值最小的**边**，一定在某个最小生成树中。

Kruskal
思想

贪心算法求解问题案例

■ 最小生成树问题—Prim算法

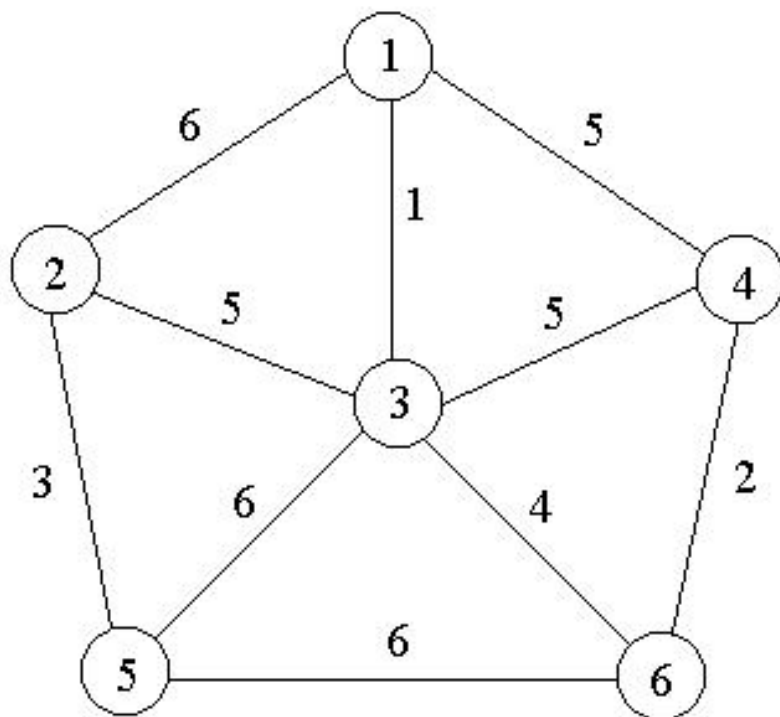
设 $G = (V, E, W)$ 是连通带权图, $V = \{1, 2, \dots, n\}$ 。

算法基本思想：首先置 $S = \{1\}$, 然后, 只要 S 是 V 的真子集, 作如下的贪心选择：**选取满足条件 $i \in S, j \in V-S$, 且 $w[i][j]$ 最小的边, 将顶点 j 添加到 S 中。**这个过程一直进行直到 $S = V$ 时为止。

在这个过程中选取到的所有边恰好构成 G 的一棵最小生成树。

贪心算法求解问题案例

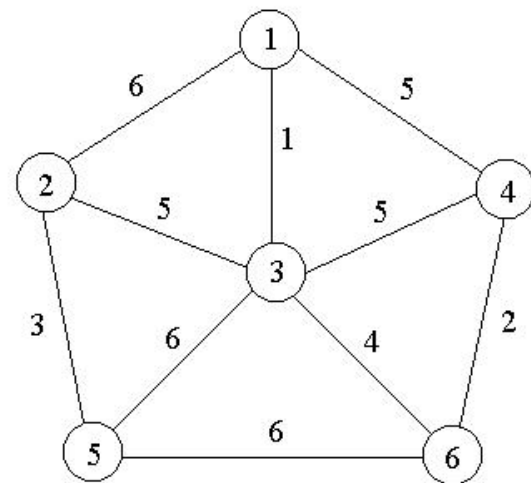
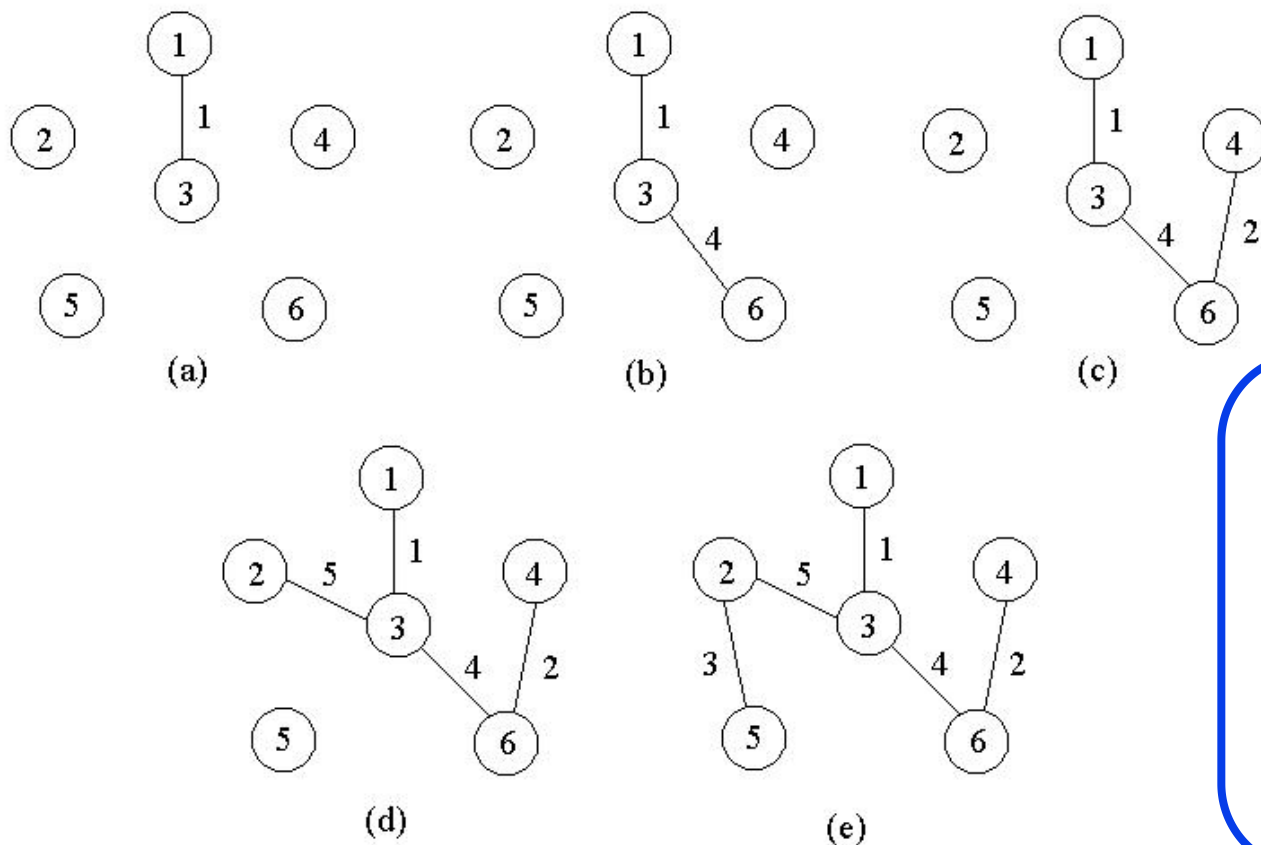
■ 最小生成树问题—Prim算法



Prim怎么求解?

贪心算法求解问题案例

■ 最小生成树问题—Prim算法



n 个节点，每次选取需要的**比较和更新**都为 $O(n)$ ，故算法所需的计算时间为 $O(n^2)$ 。

贪心算法求解问题案例

■ 最小生成树问题—Kruskal算法

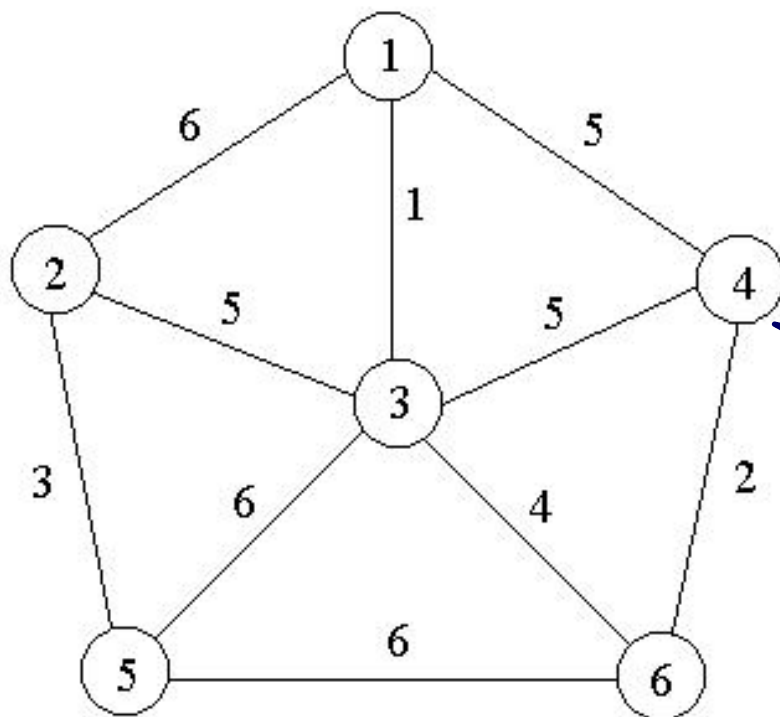
设 $G = (V, E, W)$ 是连通带权图, $V = \{1, 2, \dots, n\}$ 。

算法基本思想:

- (1) 首先将 G 的 n 个顶点看成 n 个孤立的连通分支。将所有的边按权从小到大排序。
- (2) 然后从第一条边开始, 依边权递增的顺序查看每一条边, 并按下述方法连接 2 个不同的连通分支: 当查看到第 k 条边 (v, w) 时, 如果端点 v 和 w 分别是当前 2 个不同的连通分支 T_1 和 T_2 中的顶点时, 就用边 (v, w) 将 T_1 和 T_2 连接成一个连通分支, 然后继续查看第 $k+1$ 条边; 如果端点 v 和 w 在当前的同一个连通分支中, 就直接再查看第 $k+1$ 条边。这个过程一直进行到只剩下一个连通分支时为止。

贪心算法求解问题案例

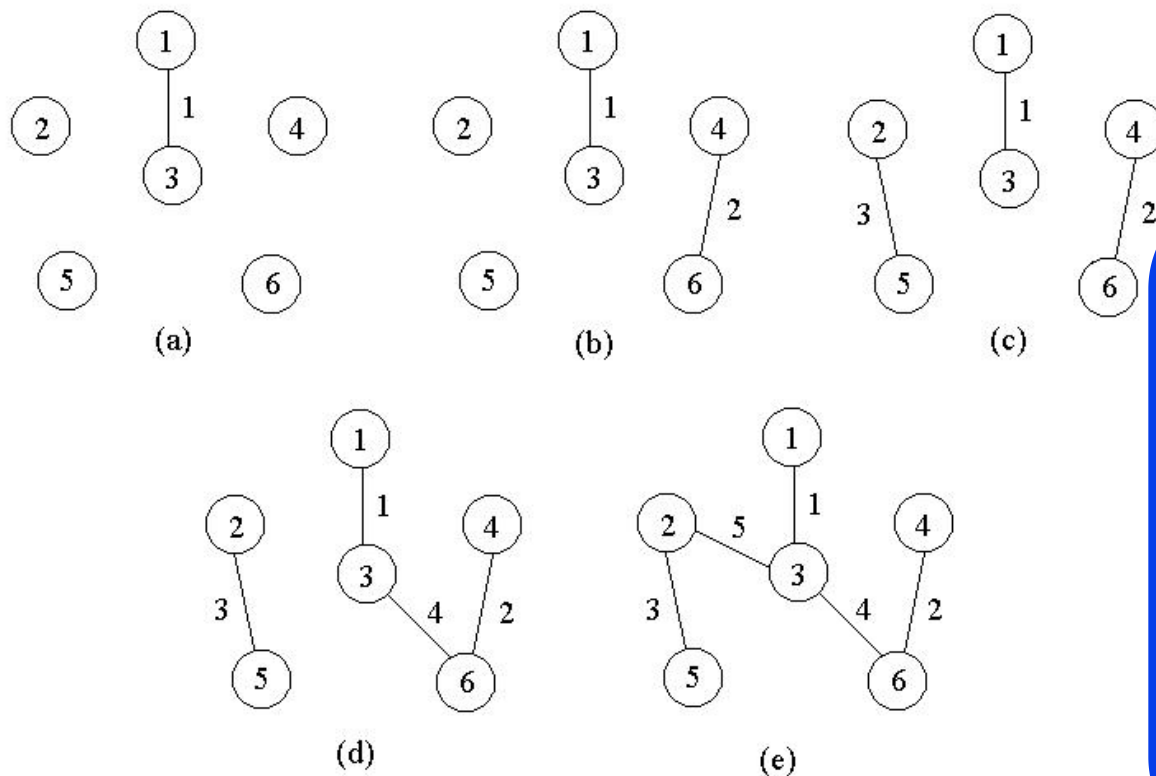
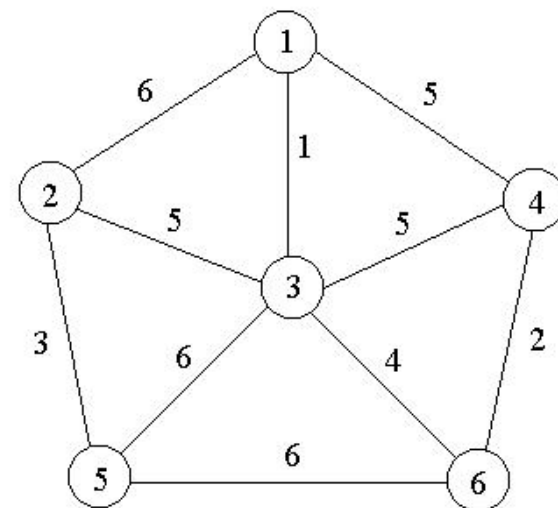
■ 最小生成树问题—Kruskal算法



Kruskal怎么求解?

贪心算法求解问题案例

■ 最小生成树问题—Kruskal算法



e 条边，排序复杂度 $O(e \log e)$ ，find与union总共需要 $O(e \log e)$ （参考算法导论 Thomas H. Cormen），故算法所需的计算时间为 $O(e \log e)$ 。

贪心算法求解问题案例

■ 最小生成树问题

当图的边数为 e 时, Kruskal算法所需的计算时间 $O(e\log e)$ 。

当 $e = \Theta(n^2)$ 时 (**边很多**) , Kruskal算法比Prim算法差, 因为:

$$e\log e = n^2\log n > n^2$$

但当 $e = o(n^2)$ 时 (**边很少**) , Kruskal算法却比Prim算法好得多, 因为:

$$e\log e = n^{2-\epsilon}\log n < n^2$$

贪心算法求解问题案例

■ 单源最短路径问题

给定带权有向图 $G = (V, E)$ ，其中每条边的权是非负实数。另外，还给定 V 中的一个顶点，称为**源**。现在要计算从源到所有其它各顶点的最短路长度。这里路的长度是**指路上各边权之和**。这个问题通常称为**单源最短路径问题**。

Dijkstra算法是代表性的解单源最短路径问题的贪心算法。

贪心算法求解问题案例

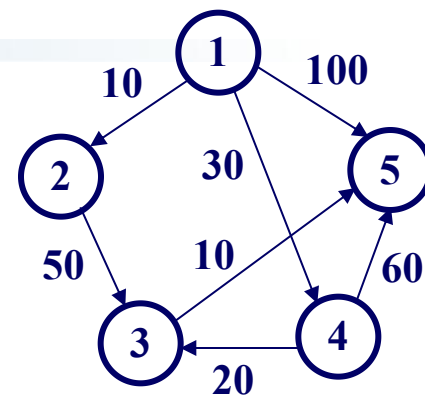
■ 单源最短路径问题—Dijkstra算法

算法基本思想：

- 设置顶点集合 S 并不断地作贪心选择来扩充这个集合。一个顶点属于集合 S 当且仅当从源到该顶点的最短路径长度已知
- 初始时, S 中仅含有源。设 u 是 G 的某一个顶点, 把从源到 u 且中间只经过 S 中顶点的路称为从源到 u 的特殊路径, 并用数组 dist 记录当前每个顶点所对应的最短特殊路径长度
- Dijkstra算法每次从 $V-S$ 中取出具有最短特殊路长度的顶点 u , 将 u 添加到 S 中, 同时对数组 dist 作必要的修改。一旦 S 包含了所有 V 中顶点, dist 就记录了从源到所有其它顶点之间的最短路径长度。

贪心算法求解问题案例

■ 单源最短路径问题—Dijkstra算法



迭代	S	u	dist[2]	dist[3]	dist[4]	dist[5]
初始	{1}	-	10	maxint	30	100
1	{1, 2}	2	10	60	30	100
2	{1, 2, 4}	4	10	50	30	90
3	{1, 2, 4, 3}	3	10	50	30	60
4	{1, 2, 4, 3, 5}	5	10	50	30	60

贪心算法求解问题案例

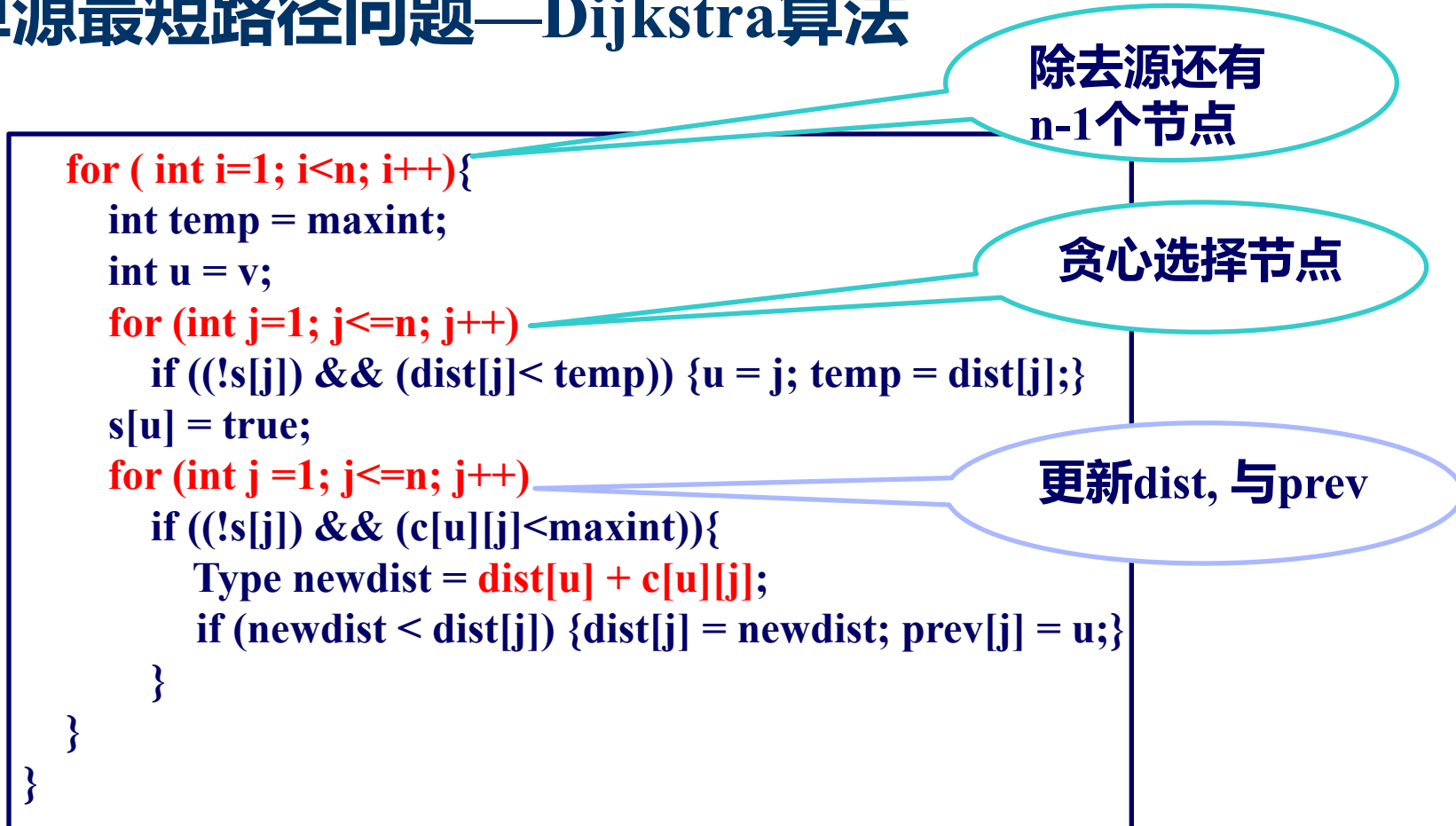
■ 单源最短路径问题—Dijkstra算法

```
template <class Type>
void Dijkstra (int n, int v, Type dist[], int prev[], Type** c) {
//n 节点数, v源节点, dist特殊路径, prev前驱节点矩阵, c邻接矩阵
    bool s[n]; //记录节点是否被选择过
    for (int i=1; i<=n; i++){ //初始化
        dist[i] = c[v][i]; //特殊路径初值为权值
        s[i] = false; //初始未被选择
        if (dist[i] == maxint) prev[i] = 0; //没有前驱
        else prev[i] = v; //前驱为源节点
    }
    dist[v] = 0; s[v] = true;
```

prev[*i*] = *j*, 表示
从 *v* 到 *i* 的最短路
径的前一个顶点
为 *j*

贪心算法求解问题案例

■ 单源最短路径问题—Dijkstra算法



贪心算法求解问题案例

- 单源最短路径问题—Dijkstra算法拓展

问：给定一个图，图中有个AGV小车，它要去图中的 k 个结点，请为该AGV规划一条代价较小的路径？

这样做是否是最短的？

贪心算法得不到最优情况的处理

■ 多机调度问题

问题描述:

设有 n 个独立的作业 $\{1, 2, \dots, n\}$, 由 m 台相同的机器进行加工处理, 作业 i 的处理时间为 t_i , 任何作业不可以拆分为更小作业, 且作业未完工前不允许中断处理。

- 多机调度问题要求给出一种作业调度方案, 使所给的 n 个作业在尽可能短的时间内由 m 台机器加工处理完成。
- 这个问题是NP完全问题, 到目前为止还没有有效的解法。对于该问题, 用贪心选择策略有时可以设计出较好的近似算法。

贪心算法得不到最优情况的处理

■ 多机调度问题—贪心策略

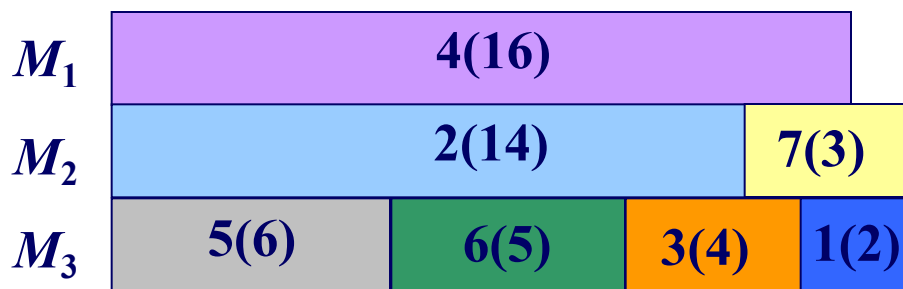
采用贪心法可以设计出解多机调度问题较好的近似算法。按**最长处理时间作业优先的贪心选择策略**策略，

- 当 $n < m$ 时，只要将机器 $1 \sim m$ 的 $[0, t_i]$ 时间区间分配给作业 i 即可，算法只需要 $O(1)$ 时间。
- 当 $n > m$ 时，首先将 n 个作业依其所需的**处理时间从大到小排序**。然后依此顺序将作业分配给空闲的处理机。算法所需的计算时间为 $O(n \log n)$ 。

贪心算法得不到最优情况的处理

■ 多机调度问题—实例

设7个独立作业{1, 2, 3, 4, 5, 6, 7}由3台机器 M_1 , M_2 和 M_3 加工处理。各作业所需的处理时间为{2, 14, 4, 16, 6, 5, 3}。按贪心策略所产生的作业调度如下图所示, 所需的加工时间为17 (**最优**)。



设5个独立作业{1, 2, 3, 4, 5}由2台机器 M_1 和 M_2 加工处理。各作业所需的处理时间分别为{10, 6, 3, 9, 8}。按贪心策略所产生的作业调度所需的加工时间为19。最优解18。

贪心算法得不到最优情况的处理

■ 硬币找零问题

假设有 n 种硬币，其重量分别为 $w_1, w_2, \dots, w_n$ ，它们的面值分别为 $v_1 = 1, v_2, \dots, v_n$ ，现需要用这些硬币付款总钱是 y ，问如何选取硬币使得付款的硬币**总重量**最轻？

$$\min \sum_{i=1}^n w_i x_i \quad \begin{cases} \sum_{i=1}^n v_i x_i = y \\ x_i \in N, i = 1, 2, \dots, n \end{cases}$$

动态规划：设 $F_k(y)$ 表示用前 k 种硬币，总钱为 y 的最小重量

$$F_k(y) = \min_{0 \leq x_k \leq \left\lfloor \frac{y}{v_k} \right\rfloor} \{F_{k-1}(y - v_k x_k) + w_k x_k\} \quad F_1(y) = w_1 \left\lfloor \frac{y}{v_1} \right\rfloor = w_1 y$$

贪心算法得不到最优情况的处理

■ 硬币找零问题—贪心策略

设 $w_1/v_1 \geq w_2/v_2 \geq \dots \geq w_n/v_n$,

使用前 k 种零钱, 总钱数为 y 时贪心法得到的总重量为 $G_k(y)$, 则有如下递推公式

$$G_k(y) = w_k \left\lfloor \frac{y}{v_k} \right\rfloor + G_{k-1}(y \bmod v_k) \quad k > 1$$

$$G_1(y) = w_1 \left\lfloor \frac{y}{v_1} \right\rfloor = w_1 y \quad (v_1 = 1)$$

$$G_1(y) = F_1(y) = w_1 y$$

$$G_k(y) \quad ? \quad F_k(y)$$

贪心算法得不到最优情况的处理

- 硬币找零问题—贪心策略

反例：3种硬币，（价值，重量）分别为 $(3, 6)$,
 $(2, 5)$, $(1, 5)$, 取价值为4。

贪心选择：选硬币1, 3, 总重量为11;

最优选择：选2个硬币2, 总重量10;

总结

- 局部最优策略的选择是贪心法的关键
- 贪心法需要正确性证明
- 如果贪心法得不到最优解，可以对问题的输入进行分析，估计算法的近似解情况
- 贪心法一般需要对原始数据预处理（排序），预处理后贪心法的时间复杂度一般较低（一阶多项式）
- 程序结构一般自顶向下，一次扫描。

作业

- 1、实现的迪斯特拉算法;
- 2、作业4-7 多处最优服务问题;

